

Parallel and Distributed Computing

Lecture 6

Distributed Information Systems

Bazam-e-Jahan

Lecturer

The University of Faisalabad

✧ Distributed information systems (DIS) are systems that manage and process information across multiple nodes (computers or servers) connected via a network. These systems are widely used in applications like **databases, enterprise resource planning (ERP), and content management systems (CMS)**. One of the key challenges in distributed information systems is **concurrency control**, which ensures that multiple transactions or operations can occur simultaneously without leading to inconsistencies or errors.

✧ Definition

- ✧ A distributed information system is a collection of interconnected nodes that work together to store, process, and manage information. The system is designed to provide **scalability**, **fault tolerance**, and **high availability**.

✧ Characteristics

1. **Decentralized:** Data and processing are distributed across multiple nodes.
2. **Transparency:** Users interact with the system as if it were a single entity, even though it is distributed.
3. **Scalability:** The system can handle increasing workloads by adding more nodes.
4. **Fault Tolerance:** The system continues to operate even if some nodes fail.

✧ Concurrency control is the process of managing simultaneous access to shared resources (e.g., data, files) in a distributed system to ensure **consistency**, **isolation**, and **correctness**. Without proper concurrency control, issues like **race conditions**, **deadlocks**, and **inconsistent data** can arise.

✧ Challenges in Concurrency Control

1. **Race Conditions:** Multiple transactions access and modify the same data simultaneously, leading to inconsistent results.
2. **Deadlocks:** Two or more transactions are stuck waiting for each other to release resources.
3. **Isolation:** Ensuring that transactions do not interfere with each other.
4. **Consistency:** Maintaining data integrity across distributed nodes.

Concurrency Control Mechanisms

✧ Concurrency control mechanisms can be broadly classified into two categories: **pessimistic** and **optimistic**.

1. Pessimistic Concurrency Control

✧ **Description:** Assumes that conflicts are likely to occur and prevents them by locking resources.

✧ **Mechanisms:**

- **Two-Phase Locking (2PL):**
 - Transactions acquire locks on resources before accessing them.
 - Locks are released only after the transaction completes.
 - *Phases:*
 - **Growing Phase:** Locks are acquired.
 - **Shrinking Phase:** Locks are released.
 - *Example:* Used in traditional relational databases like MySQL.

Concurrency Control Mechanisms

-
- **Strict Two-Phase Locking:**
 1. Locks are held until the transaction commits or aborts.
 2. Prevents cascading rollbacks.

-
- ✧ **Description:** Assumes that conflicts are rare and allows transactions to proceed without locking. Conflicts are detected and resolved at the end of the transaction.
 - ✧ **Mechanisms:**
 - **Validation-Based Protocol:**
 - Transactions are executed without locks.
 - Before committing, the system checks for conflicts.
 - If a conflict is detected, the transaction is rolled back and retried.
 - *Example:* Used in distributed databases like Apache Cassandra.

- **Timestamp Ordering:**
 - Each transaction is assigned a unique timestamp.
 - Transactions are ordered based on their timestamps.
 - Conflicts are resolved by aborting the transaction with the older timestamp.
 - *Example:* Used in distributed systems like Google Spanner.

-
- ✧ In distributed information systems, concurrency control becomes more complex due to the lack of a global clock and the need to coordinate across multiple nodes. The following techniques are commonly used:
 - ✧ **1. Distributed Two-Phase Locking (D2PL)**
 - **Description:** Extends the two-phase locking protocol to distributed systems.
 - **Mechanism:**
 - Locks are acquired and released across multiple nodes.
 - A **coordinator** node manages the locking process.
 - **Challenges:**
 - Increased communication overhead.
 - Risk of deadlocks across nodes.

✧ **Description:** Uses timestamps to order transactions across multiple nodes.

✧ **Mechanism:**

- Each node assigns timestamps to transactions.
- A global ordering is maintained to resolve conflicts.

✧ **Challenges:**

- Requires synchronization of clocks across nodes.
- High latency for global coordination.

✧ **Description:** Extends optimistic concurrency control to distributed systems.

✧ **Mechanism:**

- Transactions are executed locally without locks.
- Conflicts are detected during the validation phase.
- A global coordinator resolves conflicts.

✧ **Challenges:**

- Requires efficient conflict detection and resolution mechanisms.

✧ **Description:** Maintains multiple versions of data to allow concurrent read and write operations.

✧ **Mechanism:**

- Each transaction sees a snapshot of the data at the time it started.
- Write operations create new versions of the data.
- Old versions are garbage-collected when no longer needed.

✧ **Advantages:**

- High concurrency (readers do not block writers).
- Suitable for distributed databases.

✧ **Examples:**

- PostgreSQL, Oracle, and Google Spanner use MVCC.

✧ Distributed Databases

- ✧ **Apache Cassandra:** Uses optimistic concurrency control for high availability and scalability.
- ✧ **Google Spanner:** Uses MVCC and distributed timestamp ordering for global consistency.

✧ Enterprise Systems

- ✧ **SAP ERP:** Uses distributed two-phase locking for transaction management.
- ✧ **Oracle E-Business Suite:** Uses MVCC for concurrent access to data.

Challenges in Distributed Concurrency Control



1. **Global Coordination:** Ensuring consistency across distributed nodes without a global clock.
2. **Communication Overhead:** Managing the cost of coordinating locks or timestamps across nodes.
3. **Deadlocks:** Detecting and resolving deadlocks in a distributed environment.
4. **Scalability:** Maintaining performance as the number of nodes and transactions increases.